

COMP1009

Programming Paradigms

Lecture 00-5

Inheritance and Class Diagrams

The story so far...

- In Object Oriented Programming we decompose the program into a set of interacting objects
- **Classes define the structure of objects of that class**
- In Java we create objects (instances of a class) using the 'new' operator
 - We store a reference to the resulting object in a variable of the appropriate object reference type
 - We use the . operator to apply a method to an object (reference)
- We can use **Aggregation or Composition** to **reuse** classes within other classes (simple reuse of existing classes is a huge benefit of OO – a main reason to use it)
 - **HAS-A**: so that objects of that class **have** one of the aggregated/composed objects inside it
 - Objects which use the container don't even need to know!

Aggregation and Composition

- **Aggregation and Composition** are important OO concepts
 - They enable you to easily reuse other classes to make new classes
 - Inheritance – next lecture – is even more important/common
- Aggregation and Composition both mean that an object is **composed of**, or **aggregates**, other objects
 - E.g. it wraps the objects (has data members of that type)
 - or it ‘contains’ the objects (conceptually)
- In this case our PGPText class wraps up (and hides) the ArrayList object
 - This is aggregation/composition: ‘contains’ or ‘**has a**’ relationship
- **Aggregation**: items are contained in, or form a part of, the container item, *but have a lifetime of their own (exist beyond the container)*
 - E.g. you store a String in an ArrayList but also use it outside of the array and can use it after the ArrayList ceases to exist
- **Composition**: The composed items only last as long as the containing item lasts, *so they cease existing when container does*
 - Usually the container creates them and hides them inside it, allowing them to ‘die’ when it dies

This lecture

- We can also use Inheritance / sub-type polymorphism for re-use and adaptation
 - Inheritance is key for good object oriented design
 - This is what OO people mean when they talk about polymorphism
 - FP people usually mean parametric polymorphism when they talk about polymorphism
 - Java generics, like ArrayList, < ... > (lecture 11)
- Inheritance is one of the **most important** ideas in the object oriented paradigm
 - It underpins many other things (e.g. implementation of generics, extending classes, most design patterns)

Inheritance :

Sub-classing

Sub-type polymorphism

Creating a modified
version/type of existing classes

Reminder from lecture 1

Class	Object
Person	Jason
Animal	A specific animal
Tiger [type of animal]	A specific tiger
Car	A specific car
Ford Focus [Type!]	A specific ford focus

- You can have many different ‘instances’ of a class.
- e.g. there are many people in the UK, there are many different animals.
- Types of things (e.g. Tiger) are still ‘classes’, not ‘objects’ – they are still ‘designs’ (what a Tiger can do) rather than a specific Tiger
- With an object you should be able to say ‘this one’ and point at it

Inheritance: IS-A relationships

- A class can be a specialisation of another class
 - E.g. 4 wheeled cars, 3 wheeled cars are all types of Car



- 4 wheeled car IS-A car
- 3 wheeled car IS-A car
- Anything you can do with a car you can do with a 3 wheeled car, or a 4 wheeled car

Animal Class

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

We create a class to represent a generic animal

We don't know what 'type' of animal it is, but it contains things that are true about ALL animals

A visualisation – an array of functions?

Animal	
getType()	“Animal”
getName()	“Animal”
getNoise()	“Unknown”

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

A Bear is a type (sub-class) of Animal

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

Bear "IS-An" Animal

```
public class Bear extends Animal
{
    public String getName()
    {
        return "I am a bear";
    }

    public String getNoise()
    {
        return "GROWL...";
    }
}
```

Use 'extends' to get inheritance. Bear "IS-AN" Animal, so it gets everything in Animal and can change it or add to it

A visualisation : Animal

Animal	
getType()	"Animal"
getName()	"Animal"
getNoise()	"Unknown"

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

A visualisation : Bear

Animal	
getType()	"Animal"
getName()	"Animal"
getNoise()	"Unknown"

Animal Bear	
getType()	"Animal"
getName()	"Animal"
getNoise()	"Unknown"

```
public class Bear extends Animal
{
}
}
```

Important: we did not add any functions/methods yet!

New type: Bear

Gets everything in Animal
And can then add and change methods

A visualisation – overrides for functions

Animal	
getType()	"Animal"
getName()	"Animal"
getNoise()	"Unknown"

Bear	
getType()	"Animal"
getName()	"Bear"
getNoise()	"Growl"

```
public class Bear extends Animal
{
    public String getName()
    {
        return "I am a bear";
    }

    public String getNoise()
    {
        return "GROWL...";
    }
}
```

← Changed the behaviour
of getName and getNoise

Reminder: A Bear is a type of Animal

```
public class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public String getName()
    {
        return "I am an animal";
    }

    public String getNoise()
    {
        return "Unknown noise";
    }
}
```

```
public class Bear extends Animal
{

    public String getName()
    {
        return "I am a bear";
    }

    public String getNoise()
    {
        return "GROWL...";
    }
}
```

Bear 'inherits' the methods of Animal

- You can create Bears and (generic) Animals

```
Animal animal = new Animal();  
System.out.println("I am a(n) : " + animal.getName() );  
System.out.println("My type is : " + animal.getType() );  
System.out.println("I go : " + animal.getNoise() );
```

```
Bear bear = new Bear();  
System.out.println("I am a(n) : " + bear.getName() );  
System.out.println("My type is : " + bear.getType() );  
System.out.println("I go : " + bear.getNoise() );
```

- Here we called getType() using a Bear
 - Bear 'inherits' getType() from Animal

Bear IS-An Animal (IMPORTANT!!!)

```
Animal animal = new Animal();  
System.out.println("I am a(n) : " + animal.getName() );  
System.out.println("My type is : " + animal.getType() );  
System.out.println("I go : " + animal.getNoise() );
```

```
Animal a2 = new Bear();
```

Object reference type Animal is OK



```
System.out.println("I am a(n) : " + a2.getName() );  
System.out.println("My type is : " + a2.getType() );  
System.out.println("I go : " + a2.getNoise() );
```

- Bear IS-An Animal (it inherits methods and attributes)
 - Still uses Bear functions (not animal ones)
 - **Key: result depends upon object type not reference type**

More animals

```
public class Fish extends Animal
{
    public String getName()
    {
        return "I am a fish";
    }

    public String getNoise()
    {
        return "(glug)";
    }
}
```

```
public class Mouse extends Animal
{
    public String getName()
    {
        return "I am a mouse";
    }

    public String getNoise()
    {
        return "squeek...";
    }
}
```

Why does this work?

```
public class TestAnimals  
{
```

```
    public static void main(String[] args)  
    {
```

```
        Animal[] animals = new Animal[6];
```

← Array of object references

```
        animals[0] = new Bear();
```

```
        animals[1] = new Mouse();
```

```
        animals[2] = new Mouse();
```

```
        animals[3] = new Fish();
```

← Create objects of the extends type and store in the array

```
        animals[4] = new Mouse();
```

```
        animals[5] = new Bear();
```

```
    for ( int i = 0 ; i < animals.length ; i++ )
```

```
    {
```

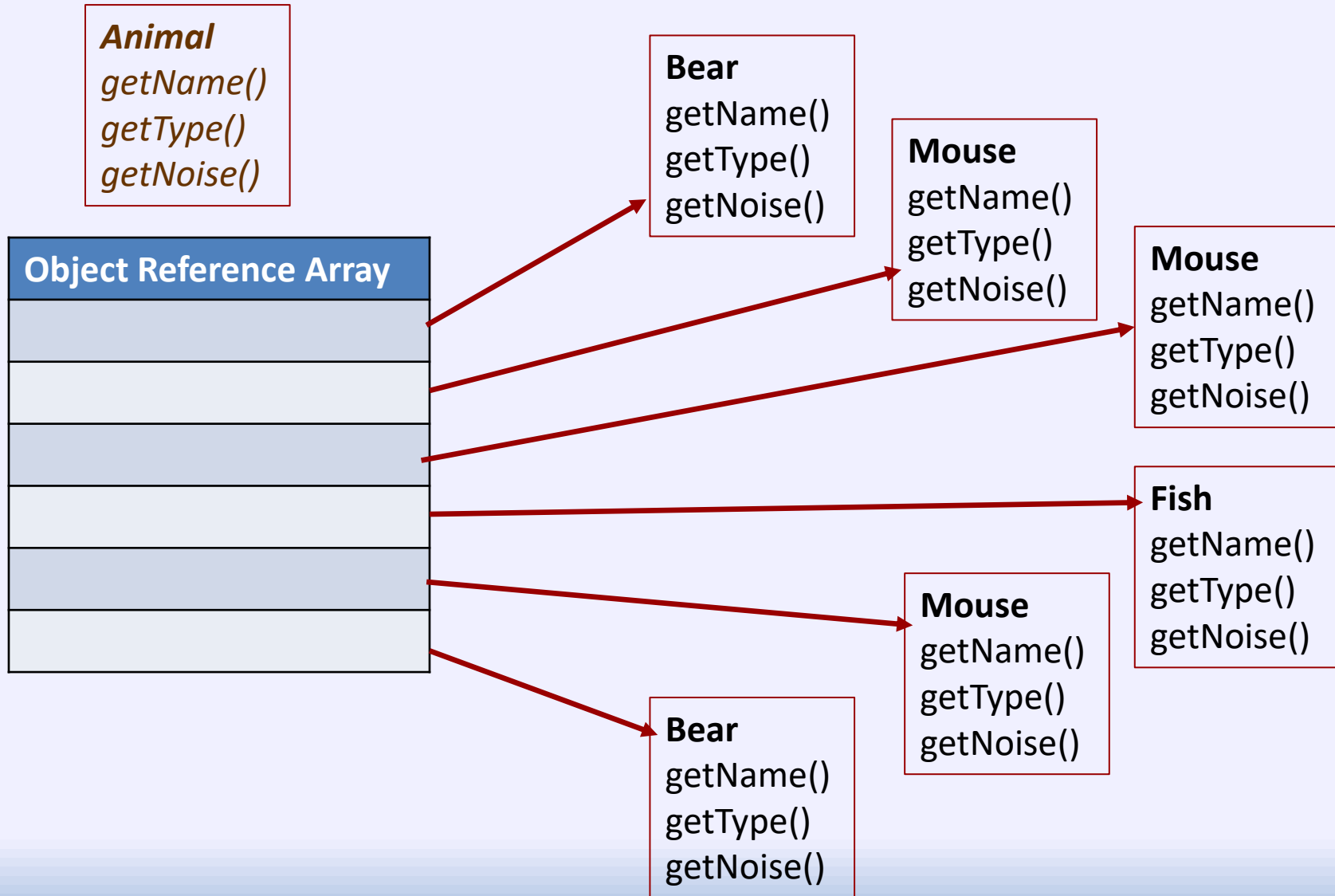
```
        System.out.println( "" + animals[i].getName() +  
                             " ... " + animals[i].getNoise() );
```

```
    }
```

```
}
```

```
}
```

6 objects and an array of object references



Sub-type polymorphism

- We used 'sub-type polymorphism' here
 - Often called Inheritance or sub-classing
 - Original class is called a “super-class” or “base class”
 - New type is called a “sub-class” or “derived class”
- We had an array of 'animal' object references
 - Remember inheritance means **IS-A**, so a Bear **IS-An** Animal
- When we called a function using the animal-type reference, the (same named) function on the object which is a specialisation of the Animal class was called
- i.e. **it knew what class the object really was** and used the function from that class instead
 - Even though the code treats it as a general Animal
- This is runtime polymorphism – the compiler does not (cannot?) know the type but looks it up at runtime

Polymorphism

- Polymorphism means “many forms”
 - not fixed to specific types
- Multiple types of polymorphism to consider
- **Parametric polymorphism:** Code works with multiple types, or regardless of types
 - What Functional Programmers mean by polymorphism
 - OO often calls it generic programming, e.g. for `ArrayList<>`
 - Java generics (e.g. `ArrayList<String>`), C++ templates
- **Ad-hoc polymorphism** (function overloading)
 - Same function name, multiple versions (differing by parameter types), e.g. `max(int,int)` vs `max(float,float)`
 - Function implementation depends upon parameter types
- **Sub-type polymorphism** (sub-classing)
 - This is usually what we mean by polymorphism in OO terms

Public, Private, Protected

Reminder of packages first

Java Packages - reminder

- Classes are organised into packages
 - Groups of classes which are similar and work together
 - These classes have special access to the methods/data of each other
- Packages can be embedded inside packages
- Directory structure must match the package structure
- Refer to classes by `packagename.classname`
 - Use the text 'import' if you access classes in other packages to avoid having to keep stating package name

Assume Animal, Bear, etc are in package animal, then this code would work:

```
animal.Animal[] animals = new animal.Animal[6];
animals[0] = new animal.Bear();
animals[1] = new animal.Mouse();
animals[2] = new animal.Mouse();
animals[3] = new animal.Fish();
animals[4] = new animal.Mouse();
animals[5] = new animal.Bear();

for ( int i = 0 ; i < animals.length ; i++ )
{
    System.out.println( "" +
        animals[i].getName() +
        " ... " + animals[i].getNoise() );
}
```

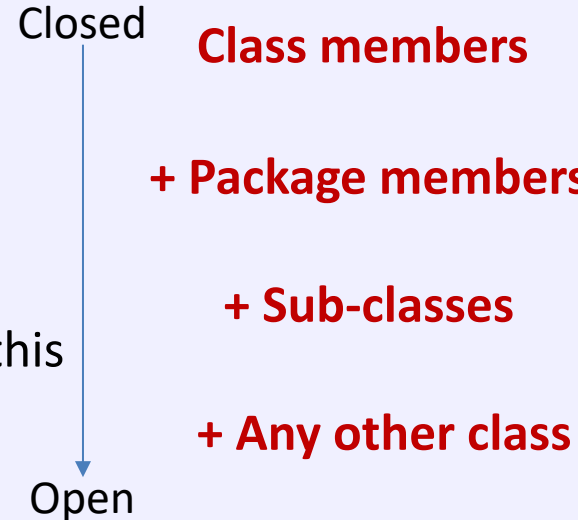
Import ...

```
import animal.*;
```

```
animal.Animal[] animals = new animal.Animal[6];  
animals[0] = new animal.Bear();  
animals[1] = new animal.Mouse();  
animals[2] = new animal.Mouse();  
animals[3] = new animal.Fish();  
animals[4] = new animal.Mouse();  
animals[5] = new animal.Bear();
```

```
for ( int i = 0 ; i < animals.length ; i++ )  
{  
    System.out.println( "" +  
        animals[i].getName() +  
        " ... " + animals[i].getNoise() );  
}
```

Access permissions

- Packages are very important because of access privileges
 - Attributes and methods are labelled with access permissions
 - **private**
 - Only this class can access this
 - ***nothing (package)***
 - This class + this package can access this
 - **protected**
 - This class + this package + subclasses can access this
 - **public**
 - Anything can access this
 - So, there is a hierarchy of importance:
 - The class itself has the most access (any modifier)
 - Then other classes in this package (can be blank, public, protected)
 - Then subclasses (can be public or protected)
 - Then 'world' (has to be public)
- 
- Closed
- Class members
- + Package members
- + Sub-classes
- + Any other class
- Open

Public, private, protected – why?

- Public: methods which are meant to be accessible outside of the class
 - i.e. things that you expect people to use to manage your objects
- Private: things that you want to hide inside your class
 - Implementation details
 - These won't be visible to other classes, so you can freely change them later
- Protected: like private, but allows subclasses (and files in same package) to change behaviour
 - Implementation details subclasses may want to use or change – you won't be able to freely change these

Permissions for overriding?

```
public class BaseClass
{
    private void privfoo1() {}
    private void privfoo2() {}
    private void privfoo3() {}

    protected void protfoo1() {}
    protected void protfoo2() {}
    protected void protfoo3() {}

    public void pubfoo1() {}
    public void pubfoo2() {}
    public void pubfoo3() {}
}
```

```
public class Subclass
    extends BaseClass
{
    public void privfoo1() {}
    protected void privfoo2() {}
    private void privfoo3() {}

    public void protfoo1() {}
    protected void protfoo2() {}
    private void protfoo3() {}

    public void pubfoo1() {}
    protected void pubfoo2() {}
    private void pubfoo3() {}
}
```

Usually keep access permissions the same on overrides in subclasses, but:

Private things are not visible to subclasses, so can reimplement a new method

If a method is visible to subclass (not private), it cannot lower the access

permissions : if it was public it stays public, if protected, can be public or protected

Next Lecture

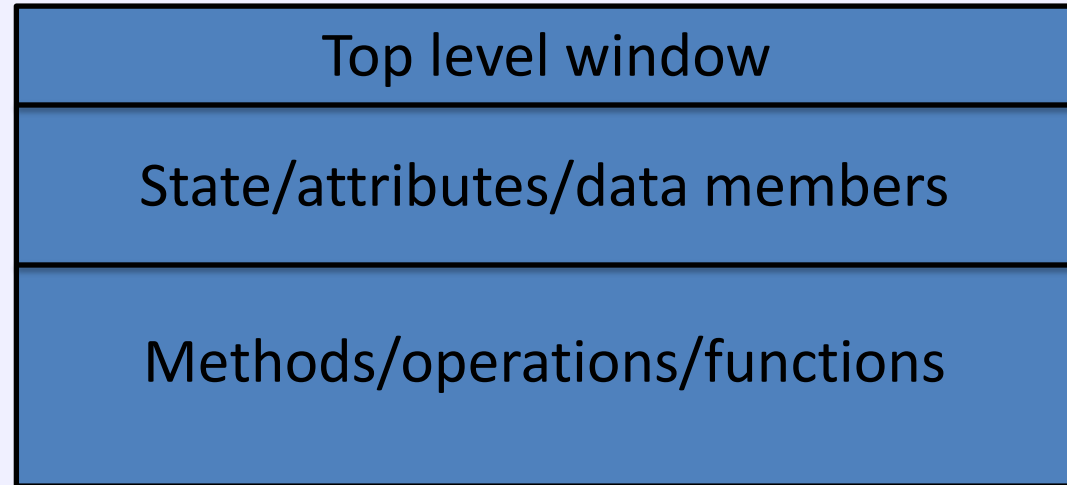
- Class diagrams
- Exceptions – an example of inheritance
- Considering PGPFFile
 - Dealing with exceptions

Class Diagrams for visualisation

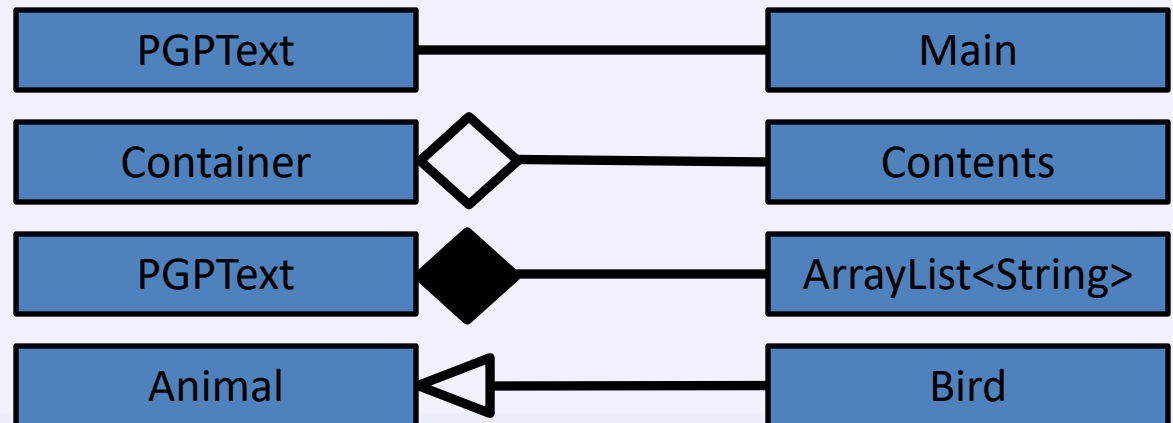
A look ahead to next week

Class diagrams - components

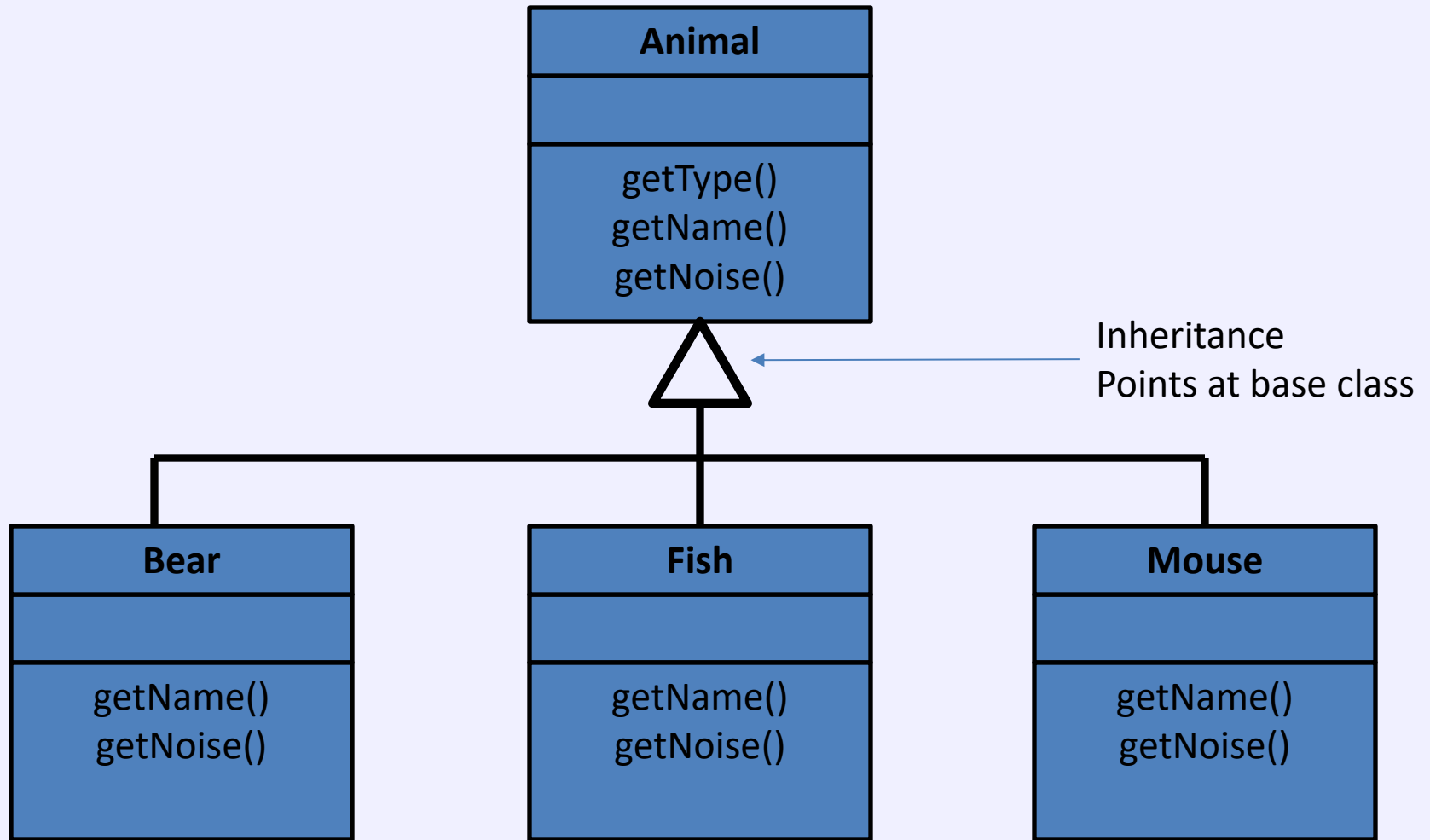
- Box for each class
 - Three sections:
 - Class name
 - Member data
 - Methods/functions



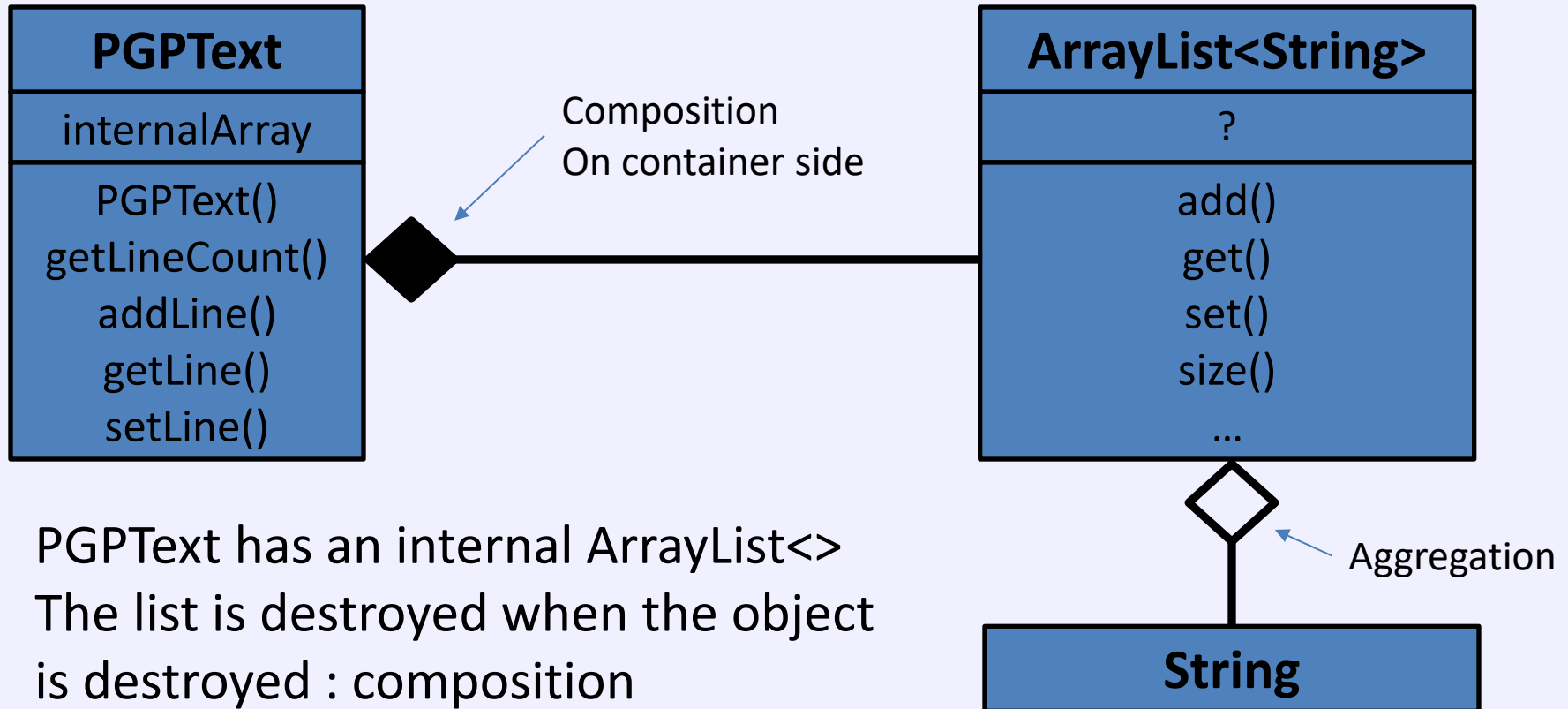
- Line for each relationship – symbol-end matters!
 - Association
 - Aggregation
 - Composition
 - Is-A / type of



Class diagram for Animals



Class diagram for PGPText



- PGPText has an internal ArrayList<>
- The list is destroyed when the object is destroyed : composition
- The ArrayList<> contains Strings
- These Strings exist outside of the ArrayList<> but references to them are added to the array: aggregation

Sometimes we omit the attributes and methods
Especially for standard classes